Curso de Especialización en IA y BigData
Sistemas de Aprendizaje Automático

Laboratorio 3: Análisis y adecuación de datos

En esta práctica trabajaremos las herramientas básicas que python nos frece para poder levar a cabo un analisis inicial de datos. Para ello, tomaremos dos dataset como referencia:

- Heart diseases (enfermedades coronarias)
- Auto MPG (Consumo de gasoline por litro de gasolina)

La presentación se llevará a cabo tomando el primero como referencia. Posteriormente, el alumno deberá llevar a cabo un análisis similar tomando los datos del segundo dataset.

Todo el codigo de la siguiente presentación se encuentra en el NoteBook adjunto.

1. Análisis y adecuación de los datos de un *dataset.* EDA

Antes de abordar cualquier problema de Machine Learning es necesario conocer bien los datos de los que se dispone.

EDA (***Exploratory Data Analysis***) es una de las primeras tareas que tiene que desempeñar el científico de datos y consiste revisar los datos que nos llegan *(por ejemplo un archivo CSV).*

Lo habitual es que tengamos un objetivo que nos hayan "adjuntado" con los datos, que indique lo que se quiere conseguir a partir de esos datos. Por ejemplo, nos pasan un dataset y nos dicen "Queremos predecir ventas a 30 días", ó "Clasificar casos malignos/benignos de una enfermedad", "Queremos identificar audiencias que van a realizar re-compra de un producto", "queremos hacer pronóstico de fidelización/abandonos de clientes", "Quiero detectar casos de fraude en mi sistema en tiempo real".

Comenzaremos leyendo la documentación adjunta al dataset y contestando las siguientes preguntas:

- ¿Cuántos registros hay?
  - ¿Nos parecen insuficientes?
  - ¿Son muchos y no tenemos Capacidad (CPU+RAM) suficiente para procesarlo?
  - ¿Están todas las filas completas ó tenemos campos con valores nulos?
  - En caso que haya demasiados nulos: ¿el resto de información es útil?

- ¿Que datos son discretos y cuales continuos?

- Si es un problema de tipo supervisado:
  – ¿Cuál es la columna de "salida" u objetivo? ¿cúal es su tipo: binaria, multiclase?
  – ¿Esta balanceado el conjunto salida?
- ¿Cuales parecen ser las *features* (características) importantes? ¿Cuales podemos descartar?
- ¿Siguen alguna distribución?
- ¿Hay correlación entre *features* (características)?
- ¿Estamos ante un problema dependiente del tiempo? Es decir un TimeSeries[15].
- ¿Cuales son los Outliers? ¿Podemos eliminarlos? ¿es importante conservarlos?
  Datos aislados que difieren drásticamente del resto y "contaminan" ó desvían las distribuciones. Por ejemplo, si la mayoría de los estudiantes sacan entre 5 y 6 en un examen, y uno obtiene un 0 o un 10, esos valores podrían considerarse outliers.

- ¿Tenemos posible sesgo de datos?
  Por ejemplo, perjudicar a clases minoritarias por no incluirlas y que el modelo de ML discrimine

Puede ocurrir que valoremos incompleto el dataset que nos ha llegado y debamos pedir a nuestro cliente/proveedor ó interesado que nos brinde mayor información de los campos, que aporte más conocimiento ó que corrija campos.

A continuación se explican diversos tipos de análisis rutinarios previos que se pueden realizar. Nos ayudaran a familiarizarnos con los datos y a entender el problema. El conjunct nos pueden ayudar a eliminar, corregir o completar datos erróneos o incompletos. Estas actividades se suele denominar análisis y limpieza de datos (***data analysis* y *data cleansing***).

Para este ejercicio se utilizarán dos *datasets* disponibles en el *UCI Machine Learning Repository*.

- El *dataset* de enfermedades del corazón (heart diseases) está disponible en

  https://archive.ics.uci.edu/dataset/45/heart+disease

  Este es un *dataset* para clasificación. El atributo *target u objetivo* representa la presencia de estrechamiento de vasos sanguíneos del corazón representado por un valor entre 0 y 3.

- El *dataset* para MPG (*Miles per Gallon* o millas recorridas por galón de gasolina) está disponible en

  https://archive.ics.uci.edu/dataset/9/auto+mpg

  Este es un *dataset* para regresión. El atributo *target* representa el número de millas que cierto automovil con determinadas características puede recorrer con un consumo de un galón de gasolina.

DataSet *Heart Disease* (Enfermedades del corazón)



Ilustración: Información general del dataset de enfermedades del corazón

Es un dataset para clasificación y lo utilizaremos para predecir el padecimiento de una enfermedad cardiaca.

Hay disponibles cuatro *datatsets* distintos, con la misma estructura (los mismos atributos), pero para cuatro instituciones distintas: Budapest (Hungría), Cleveland (EEUU), Long Beach (EEUU) y Zurich (Suiza). La base de datos de Cleveland es la que se ha utilizado más frecuentemente, y a ella corresponde el número de instancias (303) indicado en la ilustración anterior.

Hay 75 atributos, de los cuales solo utilizaremos 14 que son decritos en la siguinte tabla. De ellos, 13 como *features* (atributos predictores) más el atributo *target* u objetivo.

- La documentación del *dataset* está en el fichero heart-disease.names.
- Los datos del dataset de Cleveland están en el fichero processed.cleveland.data.

Todos los valores son numéricos, pero como ya se ha explicado, eso no significa que todos los atributos sean numéricos (pueden venir como cadenas de texto que representas valores numéricos).

Se puede distinguir entre atributos numéricos y categóricos consultando el significado del campo y sus diferentes valores en la documentación y en los propios datos. El último atributo (num) es el ***target o atributo que hay que predecir***. Este toma cuatro valores posibles (0, 1, 2 y 3). Pero solo el primero (0) indica la ausencia de enfermedad del corazón. Más adelante se verá cómo se cambian los valores 1, 2 y 3 por el valor 1, para así reducirlo a un problema de clasificación binaria:

(0- no padece enfermedad, 1-padece enfermedad).

#	Nombre	Significado	Tipo	Posibles valores
1	**age**	Edad en años	Numérico (entero)	
2	**sex**	Sexo	Categórico (dicotómico)	0: mujer 1: hombre
3	**cp**	Tipo de dolor en el pecho	Categórico	1: *typical angina* 2: *atypical angina* 3: *non-anginal pain* 4: *asymptomatic*
4	**trestbps**	(Presión arterial)	Numérico (entero)	
5	**chol**	(Colesterol)	Numérico (entero)	

6	**fbs**	(Glucosa en sangre en ayuno)	Categórico (dicotómico)	
7	**restecg**	(Resultado de electrocardiograma)	Categórico	0: *normal* 1: *Con anormalidad de la onda ST-T* 2: *hipertrofia ventricular izquierda*
8	**thalach**	(Máximo de pulsaciones por minuto)	Numérico (entero)	
9	**exang**	(Se ha inducido Angina por ejercicio)	Categórico (dicotómico)	1: *Yes* 2: *No*
10	**oldpeak**	*Depresión del segmento ST inducida por el ejercicio en relación con el reposo*	Numérico (real)	
11	**slope**	*Pendiente del segmento ST en el pico del ejercicio*	Categórico	1: *ascendente* 2: *plano* 3: *descendente*
12	**ca**	*Número de vasos principales (0-3) coloreados por fluoroscopia*	Numérico (entero)	Valores de 0 a 3
13	**thal**	*thal o prueba de estrés con talio*	Categórico	3: *normal* 6: *defecto fijo* 7: *defecto reversible*
14	**num**	(Diagnóstico de enfermedad cardíaca: estado de enfermedad angiográfica) Indica si las arterias coronarias presentan estenosis significativa según su diámetro.	Numérico (entero). Valores 0 a 3. Se convertirá a dicotómico: 0 vs 1, 2 ó 3.	0: estrechamiento del diametro < 50% 1: estrechamiento del diametro > 50%

DataSet *Auto MPG o Miles per Gallon* (Consumo de gasolina medido en millas por galón)

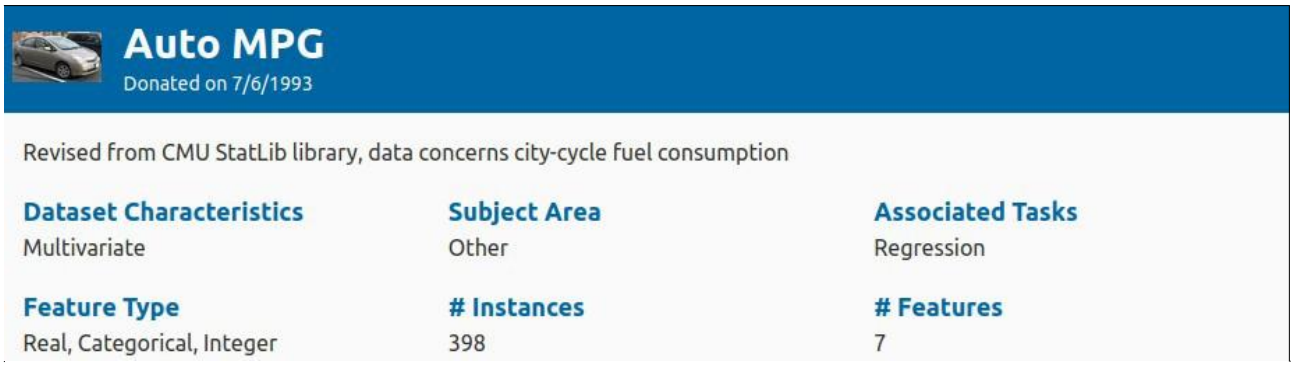


Ilustración: Información general del dataset de consumo de gasolina en millas por galón

Es un dataset para regresión. Se trata de predecir el número de millas que recorre un automóvil con un galón de gasolina.

Hay 9 atributos. Todos toman valores numéricos, excepto uno que *contiene el modelo del coche*. Este es completamente irrelevante y por tanto se puede eliminar. Quedan 8 atributos. ***El atributo target u objetivo es mpg. Los otros 7 son los atributos predictores (features).***

- La documentación del *dataset* está en el fichero auto-mp.names.
- Los datos están en el fichero ***auto-mpg.data***. Como se explica en la documentación, la diferencia con el fichero ***auto-mpg.data-original*** es que de este último se han eliminado 8 instancias que tenían un valor desconocido para el atributo mpg.

#	Nombre	Significado	Tipo	Posibles valores
1	**mpg**	Millas que se recorren con un litro de gasolina.	Numérico (real)	
2	**cylinders**	Número de cilindros	Numérico (entero)	
3	**displacement**	Desplazamiento	Numérico (real)	
4	**horsepower**	Potencia en caballos de vapor	Numérico (real)	
5	**weight**	Peso	Numérico (real)	
6	**acceleration**	Aceleración	Numérico (real)	
7	**model year**	Año del modelo	Numérico (entero)	
8	**origin**	Lugar del fabricante	Categórica	1: USA 2: Europe 3: Asia
9	**car name**	Nombre del coche (modelo)	Categórica	

1.1. Análisis con hoja de cálculo

No está de más cargar el fichero con un programa de hoja de cálculo, y verificar en particular los diferen- tes valores para atributos categóricos. Es especialmente recomendable el programa de hojas de cálculo de LibreOffice (LibreOffice Calc). Tiene multitud de opciones para importación de datos en ficheros en formato CSV.

Se puede cargar el fichero processed.cleveland.data con un programa de hoja de calculo (LibreOffice Calc o Microsoft Excel). El programa detecta el formato del fichero CSV y ofrece opciones por defecto para la importación y una previsualización del resultado con las opciones seleccionadas.

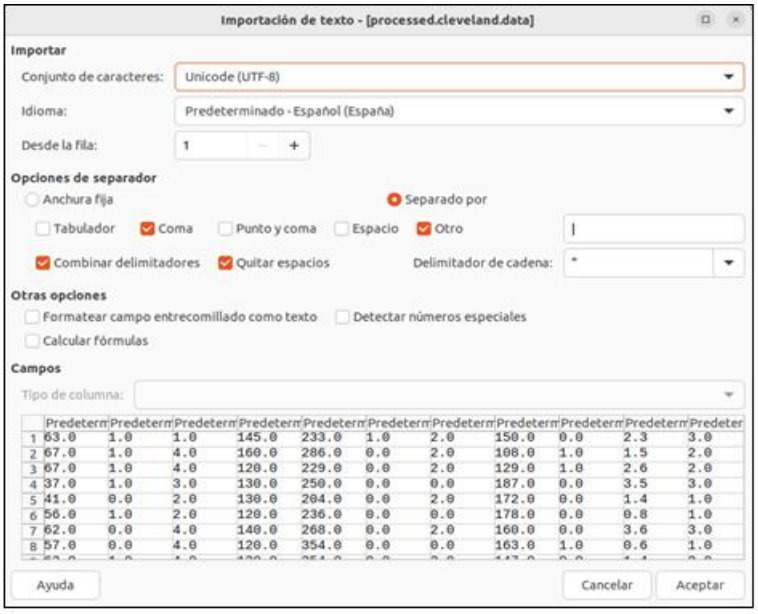


Ilustración: Importación de fichero en formato CSV con la hoja de cálculo Calc de OpenOffice

Una vez importado el fichero, el filtro automático permite mostrar listas de los valores existentes para cada atributo. Se activa con el menú “Datos”, opción “Filtro automático”. Por ejemplo, se puede verificar que para el tercer atributo (cp), que es categórico, los valores presentes son 1, 2, 3 y 4. Para los atributos numéricos se muestra una lista más larga, con todos los valores presentes.

1.2. Carga y análisis inicial de datos con Python

La carga de los datos para el *dataset* de las enfermedades del corazón se puede hacer con el siguiente código en Python.

```
import pandas as pd


# Añadimos el nombre de las columnas
df =  pd.read_csv('./heart/processed.cleveland.data',
    names=['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach', 'exang', 'oldpeak', 'slope', 'ca', 'thal',  'num']
)
```

Después de cargar los datos los examinamos.

```
print(df.shape)  # Cuantas filas y columnas tiene el dataset
print(df.dtypes) # Tpo de dato de cada columna (caracteristica o atributo)
print(df.head(20).to_string()) # Devuelve las 20 primeras filas sin cortes tabulares (todo en una sola línea)
print(df.describe()) # Presenta los valores estadisticos
```

1.3. Detección y sustitución de valores no numéricos en atributos numéricos

Se puede ver en el fichero, y en la documentación del *dataset*, que todos los atributos tienen valores numéricos. El tipo de Python para todas las columnas es float64, excepto para las dos últimas. **¿Por qué?** Probablemente porque los valores nulos existentes, según lo que se indica en la documentación del *dataset* para *missing values*, están en estas columnas y por este motivo (contienen valores nulos) automaticamente dejan de aparecer como caracteristicas de tipo numérico.

Ahora se trata de encontrar dónde están estos valores. El siguiente código muestra las filas que tienen un valor no numérico en las columnas ca y thal:

```
df[pd.to_numeric(df.ca, errors='coerce').isna() | pd.to_numeric(df.thal, errors='coerce').isna()]
```

Con esta operación obtenemos lo siguiente:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num
87	53.0	0.0	3.0	128.0	216.0	0.0	2.0	115.0	0.0	0.0	1.0	0.0	?	0
166	52.0	1.0	3.0	138.0	223.0	0.0	0.0	169.0	0.0	0.0	1.0	?	3.0	0
192	43.0	1.0	4.0	132.0	247.0	1.0	2.0	143.0	1.0	0.1	2.0	?	7.0	1
266	52.0	1.0	4.0	128.0	204.0	1.0	0.0	156.0	1.0	1.0	2.0	0.0	?	2
287	58.0	1.0	2.0	125.0	220.0	0.0	0.0	144.0	0.0	0.4	2.0	?	7.0	0
302	38.0	1.0	3.0	138.0	175.0	0.0	0.0	173.0	0.0	0.0	1.0	?	3.0	0

Ilustración: Valores no numéricos en un Dataframe

Atención

No supone un problema el mero hecho de que existan valores no numéricos en el `Dataframe` en el que se carga un *dataset*. Sí supone un problema que existan en un atributo que, de acuerdo con su definición, es numérico. Los valores no numéricos en atributos numéricos suelen representar valores nulos o desconocidos, o pueden deberse a errores en la preparación del *dataset*. En cualquier caso, hay que detectarlos. Ahora veremos cómo convertirlos en valores nulos, a la vez que se cambia el tipo del atributo a un tipo numérico.

Se puede ver que los valores nulos están representados en el fichero CSV con el carácter '?'.

IMPORTANTE. *Una cosa son los valores nulos y otra los valores no numericos en atributos numericos. Si un atributo de tipo numerico incluye un character como "?" en uno de sus campos como consecuencia de no haber introducido en su momento valor alguno, el sitema etiqueta el atributo cono no numérico.*

El siguiente código convierte a valores NaN todos los valores no numéricos de los atributos numéricos (en este caso en los atributos ***ca*** y ***thal***), a la vez que cambia el tipo de estos atributos a numéricos.

```
df.ca = pd.to_numeric(df.ca, errors='coerce')
df.thal = pd.to_numeric(df.thal,  errors='coerce')
```

Pregunta: ¿Porqué es necesario cambiar el tipo de atributo después de sustituir todos los valores no numericos por NaN?

Ahora, con **df.info()** se puede verificar el tipo de todos los atributos y podemos ver el número de valores nulos que hay para cada uno. Se puede verificar también que estos valores nulos son valores NaN. Una vez que los tipos de todos los atributos son los correctos, se procede a ***eliminar o sustituir*** los valores nulos en atributos numéricos.

NOTA: primero debemos cambiar los valores no numericos por NaN. Después decidimos si eliminamos o sustituimos estos valores nulos.

1.4. Eliminación o sustitución de valores nulos

Caben varias alternativas:

1. **Eliminar las filas que contienen valores nulos**. Esta es una solución sencilla pero demasiado drástica porque se pierden datos.
   - Se pueden borrar del df las filas que tienen un valor nulo en un atributo (por ejemplo para ***talla***)

```
df.dropna(subset=['talla'],  inplace=True)
```

   Con el atributo inplace=Flase, se mantene df y se devuelve un df sin la fila que tenían el atributo a nulo:

```
df.dropna(subset=['talla'],  inplace=False)
```

   - Se pueden eliminar las filas en un df que tienen un valor nulo ***en cualquier atributo*** con:

```
df.dropna(inplace=True)
```

   La siguiente instrucción devuelve un nuevo df sin las filas pero mantiene el df original :

```
Nuevo_df=df.dropna(inplace=False)
```

2. **Eliminar atributos que contienen valores nulos**. Es una solución aún más drástica, y en general más indeseable pero podría ser aconsejable para atributos que contienen demasiados valores nulos. Por ejemplo, se podría eliminar el atributo ***talla*** con:

```
df.drop(['talla'], axis=1,  inplace=True)
```

   Hay que valorar las consecuencias de borrar datos. El peor escenario posible sería cuando hay un porcentaje notable de filas con valores nulos y están en columnas cuyo valor es relevante para el modelo que se está desarrollando. Entonces conviene considerar alguna de las siguientes opciones.

3. **Sustituir el valor nulo por otro valor representativo**. Por ejemplo, si el nombre del atributo es ***age***, se podrían sustituir los valores nulos por la media aritmética de los valores de esa columna de la siguiente forma:

```
df['age'].fillna(df['age'].mean(),  inplace=True)
```

4. **Sustituir los nulos por un valor concreto**. Si en la columna en la que aparecen hay valores numéricos, se podrían sustituir por 0. Se podría asignar 0, por ejemplo, al atributo ***age*** en todas las filas en las que tiene valor nulo con:

```
df['age'].fillna(0,  inplace=True)
```

   Se podrían sustituir todos los valores nulos por 0 de la siguiente forma. Esto es muy drástico, porque sustituye valores nulos en todas las columnas.

```
df.fillna(0, inplace=True)
```

1.5. Revisión del tipo del atributo objetivo o *target*

En el dataset de enfermedades del corazón (Heart Disease), podemos observar que el atributo objetivo ***num*** se ha declarado con el tipo de dato int64, es decir, como una variable numérica de valores enteros. Sin embargo, aunque sus valores estén representados mediante números, esto no significa necesariamente que debamos interpretarlos como cantidades numéricas continuas.

En este dataset, el atributo ***num*** representa el diagnóstico de enfermedad cardíaca. Sus posibles valores son:

0: ausencia de enfermedad cardíaca.

1, 2, 3 y 4: distintos niveles de presencia de enfermedad cardíaca.

Por tanto, si deseamos utilizar este dataset para abordar un problema de clasificación multiclase debemos interpretar estos valores como etiquetas de clase y no como cantidades numéricas. Esto lo podemos llevar a cabo mediante la siguiente instrucción.

```
df['num']=pd.Categorical(df['num'])
```

Después de ejecutar esta instrucción, el atributo ***num*** dejará de tener el tipo int64 y pasará a tener el tipo category ***(valores posibles ahora para el atributo num: 0, 1, 2, 3 o 4).***

Hay que tener en cuenta que en un problema de clasificación, el objetivo del modelo consiste en predecir a qué clase pertenece cada observación y no en estimar una cantidad numérica.

Para el *dataset* del consumo de gasolina (MPG), se puede ver en la documentación que el atributo *target* u objetivo, ***mpg***, es de tipo float64, es decir, de tipo numérico.

Puede ocurrir que al cargar el dataset algunos valores de una columna numérica aparezcan almacenados como cadenas de texto (object o string), por ejemplo, cuando existen valores desconocidos representados mediante símbolos como ? o cuando los números se han leído como texto.

Para asegurarnos de que la columna ***mpg*** contiene valores numéricos, podemos utilizar la siguiente instrucción:

```
df.mpg = pd.to_numeric(df.mpg,  errors='coerce')
```

Si el valor del campo no es numeric y por tanto no lo puedo convertir, el argumento *errors='coerce'* genera un NaN en su lugar. De esta forma aseguramos que el atributo pueda ser convertido a tipo numeric y que, en caso de conteer valores nulos, estos quedan representados por NaN.

1.6. Análisis de la distribución de frecuencias para atributos

Una vez que se han eliminado valores nulos en atributos numéricos, y que los tipos de todos los atributos del Dataframe son correctos, conviene hacerse una idea de cuales son ciertas caracteristicas de los datos a traves de un análisis de frecuencias.

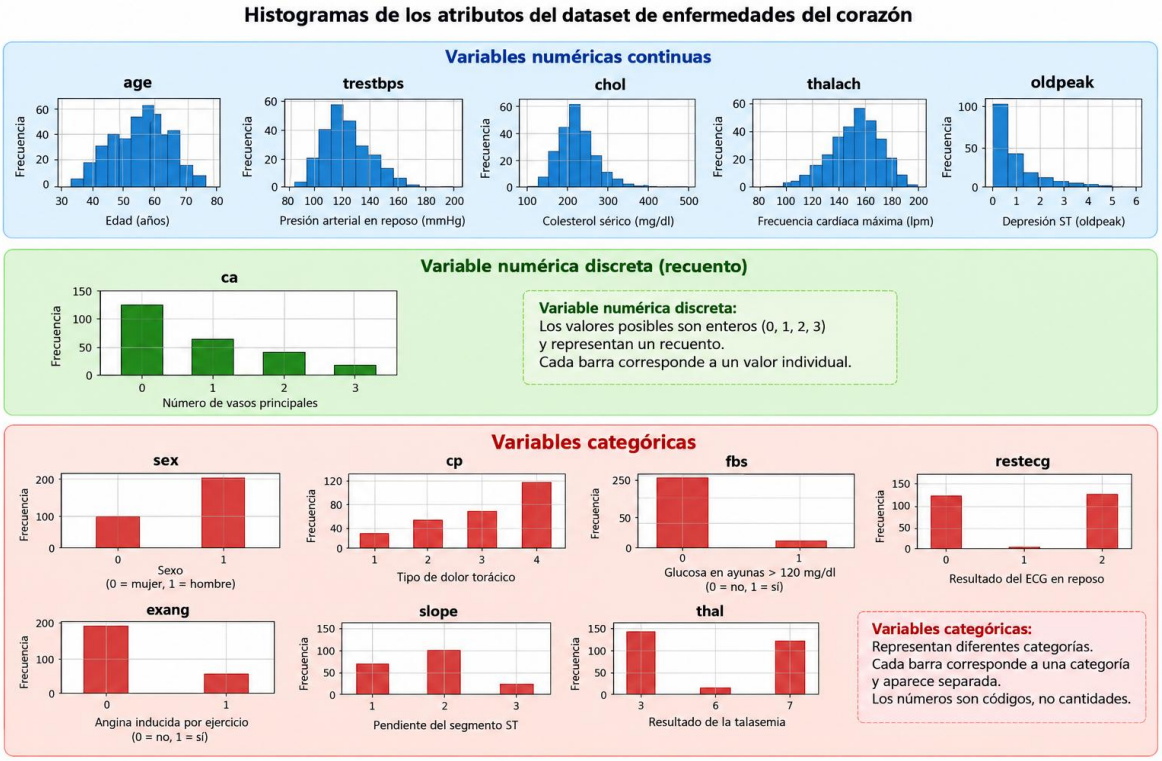
Un instrumento fundamental para el análisis de los valores de los atributos es el ***histograma***. Un histograma ***muestra la frecuencia para cada valor del atributo***. Se generan de manera distinta dependiendo de si son para atributos categóricos o numericos (int o float).

Un histograma para ***un atributo categorio*** mostrará una sucesión de barras que representan la frecuencia de cada valor que toma el atributo.

Un histograma para ***un atributo continuo*** mostrará igualmente una sucesión de barras pero ahora, cada barra representa la ***frecuencia de un rango*** de valores del atributo.

Por ultimo, ***otra posibilidad*** es el ***histograma de un atributo numerico con valores discretos***. En ese caso, ocurre algo parecido al primer caso. Cada barra representa la frecuencia de cada valor dicreto del atributo.

La diferencia es la interpretación que se dá en cada caso.



1.6.1. Histogramas para atributos categóricos vs diagrama de barras

El atributo ***num*** para enfermedades del corazón se ha convertido en un paso previo en atributo categórico, porque es el atributo *target* u objetivo de un problema de clasificación. Por ser el atributo ***objetivo o target***, es especialmente importante el análisis de sus valores y de sus respectivas frecuencias.

Su lista de valores, con la frecuencia absoluta para cada uno, se puede obtener con:

```
df['num'].value_counts()
```

Se puede obtener esta lista de frecuencias ordenadas por los valores con:

```
df['num'].value_counts().sort_index()
```

Se puede mostrar un **diagram de barras** para estos datos con:

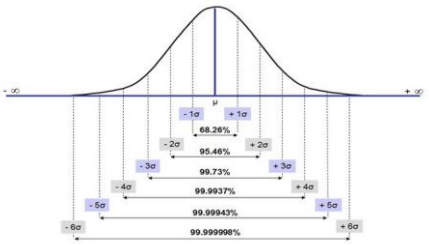
```
df['num'].value_counts().sort_index().plot(kind='bar')
```

En general, para representar un atributo categórico utilizamos un ***diagrama de barras*** pero si el atributo, siendo de tipo categórico almacena valores numericos para codificar las clases, podríamos representarlo con un histograma con la ayuda de la función ***df.hist()***

1.6.2. Histogramas para atributos con valores numéricos

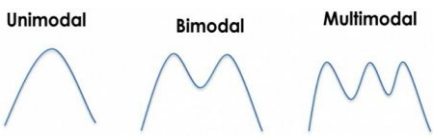
Recordatorio: distribución normal de atributos

Es muy frecuente que las frecuencias de los distintos valores para atributos numéricos se aproximen a una distribución normal.
En una distribución normal, el valor más frecuente es la media (μ en la ilustración). La frecuencia de los valores disminuye gradualmente con - forme estos están más alejados de la media. La velocidad con la que de - crecen depende de la desviación típica σ, y es menor para valores gran- des de σ.

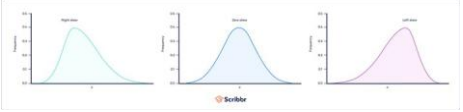


Los ingresos de una población, por ejemplo, suelen seguir una distribución normal, especialmente en países desarro - llados y sin grandes desigualdades sociales. La mayor frecuencia es para ingresos cercanos a la media. La frecuen - cia de los valores particulares de los ingresos disminuye conforme estos se alejan más de la media, por arriba o por abajo.

Es también frecuente que la distribución sea bimodal. Por ejemplo: la altura de la población de un país sigue una distribución diferente para hombre y para mujeres. La estatura de hombres y de mujeres por separa - do sí sigue una distribución normal monomodal. Pueden haber también distribuciones multimodales.



También son frecuentes las distribuciones normales inclinadas (*skewed*), hacia la derecha o hacia la izquierda.

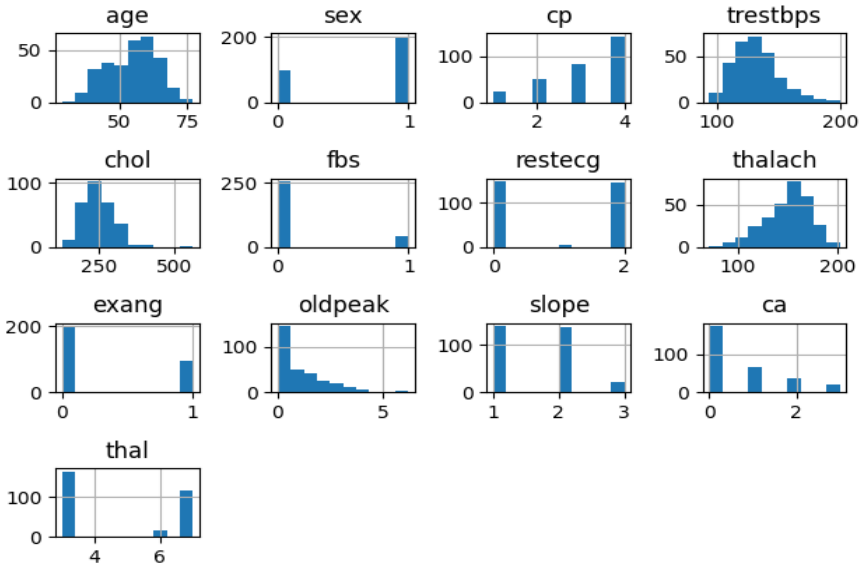


El método ***hist*** de la clase DataFrame muestra el histograma de todos los atributos que toman valores numéricos (sean de tipos ***int*** ***o float*** o sean del tipo ***category*** que contengan valores numéricos para codificar las distintas categoría).

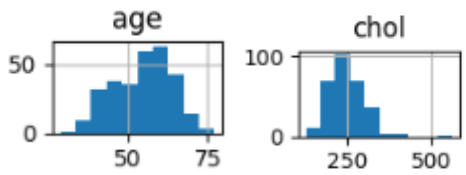
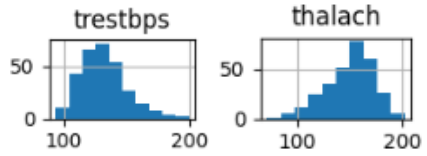
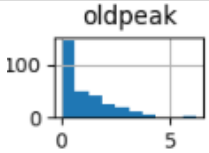
```
df.hist()
plt.tight_layout()
```

Nota: plt.tight_layout() hace que no se solapen los histogramas, mejorando así la presentación.

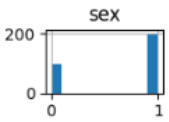
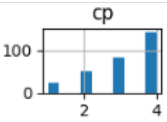
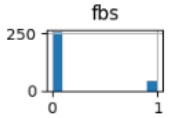
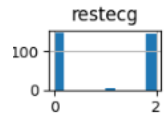
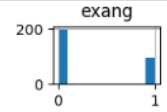
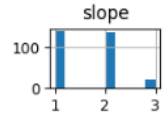
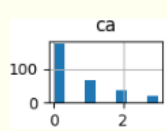
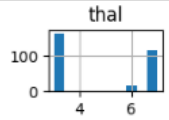
Analicemos los hostogramas:



Histogramas para atributos con valores numéricos, con df.hist()

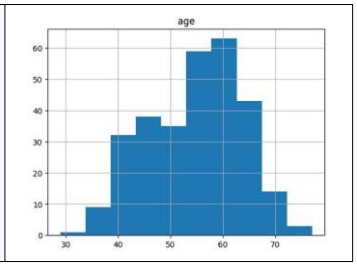
	Distribución normal. La distribución de age podría ser bimodal (dos senos).
	Distribución normal inclinada.
	Distribución normal muy inclinada.

Tambien podemos encontrar otros atributos que no siguen una distribución normal, o para los que no está tan claro. Tienen pocos valores y en principio, a juzgar solo por el histograma, parece que todos ellos son categóricos.

	Atributo categórico dicotómico, el sexo.
	Atributo categórico, el tipo de dolor en el pecho, con cuatro posibles valores.
	Atributo categórico dicotómico, que indica si la glucosa en sangre en ayuno (*fasting blood sugar*) es mayor de 120 mg/dl.
	Atributo categórico con tres posibles valores.
	Atributo categórico dicotómico.
	Atributo categórico con tres posibles valores.
	Atributo numérico de tipo entero, con posibles valores 0, 1, 2 y 3, y que indica el número de vasos importanes coloreados por fluoroscopía
	Atributo categórico con tres posibles valores.

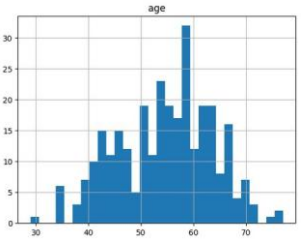
Podemos obtener el ***histograma de atributos particulares*** o para ***un grupo de ellos***, siempre que sus valores sean numéricos. Por ejemplo:

```
df.hist(['age'])
```

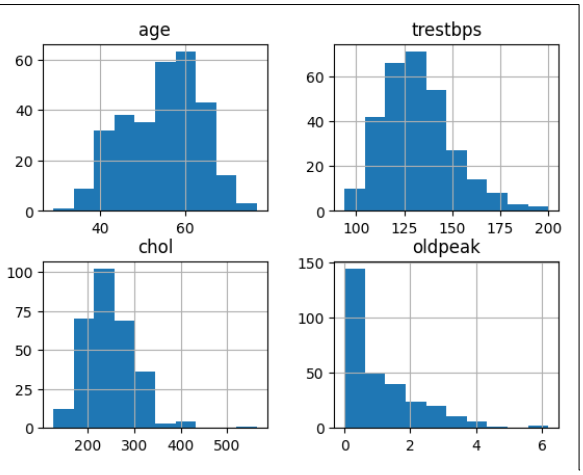


Nota. Con el parámetro **`bins`** se puede especificar el número de intervalos en que se divide el rango de valores para el atributo. Cuanto mayor su valor, más intervalos se muestran y más pequeños son.

```
df.hist(['age'], bins=30)
```

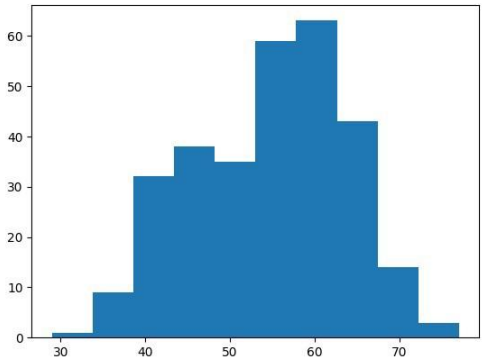


```
# Mostramos el histograma de algunos atributos
df.hist(['age','trestbps','chol','oldpeak'])
```



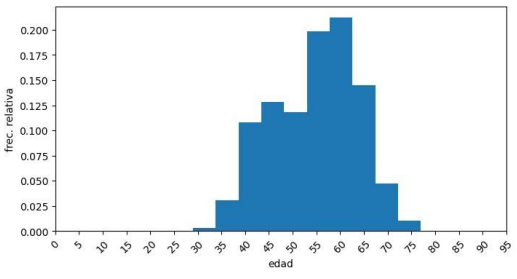
En el siguiente ejemplo añadimos etiquetas a los ejes vertical y horizontal, establecemos el rango y los saltos en el eje horizontal, el tamaño del grafico y la inclinación de las marcas (*ticks*) de valor en el eje horizontal.

```
plt.hist(df['age'])
```



Una alternativa es utilizar la función hist de ***matplotlib.pyplot*** que ofrece un mayor control de la apariencia y de opciones de visualización.

```
plt.hist(df['age'])
plt.xlabel('edad')
plt.ylabel('frec. absoluta')
plt.xticks(np.arange(0,100,5))
plt.gcf().set_size_inches(8,  4)
plt.gca().tick_params(axis='x',  labelrotation=45)
```



Actividad 1:

Comenta el resultado de ejecutar la instrucción ***df.describe()*** del punto 1.2 de este boletin de prácticas. ¿Que información útil obtenemos para el proceso que estamos llevando a cabo ***(análisis y limpieza de datos)***?

Actividad 2:

a) Tomando como referencia el notebook correspondiente al Lab03 que contiene el código utilizado en esta presentación utilizado para analizar el dataset de enfermedades del corazón, haz una copia y modificarlo para llevar a cabo las mismas operaciones y análisis pero ahora importando el dataset AutoMPG.

   a. Los datos del dataset AutoMPG se encuentran en el archive “auto-mpg.data”
   b. El nombre de las columnas o atributos se encuentra en el archive “auto-mpg.names”

b) Una vez obtenidos los histogramas (y diagramas de barras que correspondan) añade a continuación un analisis de estos de la misma forma que se ha hecho en el boletin de práctices para los histogramas correspondientes al dataset de enfermedades del corazón ***(Heart Disease)***